TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROGRAMACIÓN DE BASE DE DATOS

TAREA 9: PROGRAMACIÓN DEL LADO DEL SERVIDOR: DISPARADORES (TRIGGERS)

PROFESORA:

ALEJANDRA GONZALEZ SÁNCHEZ

NOMBRE:

MONTERRUBIO LÓPEZ LUIS GAEL

GRUPO: 5701

FECHA: 30-10-26

1. Introducción y objetivo

Este informe explica qué son los disparadores (triggers) en una base de datos relacional, cómo funcionan, sus tipos y en qué escenarios conviene usarlos. Después describe el diseño, la implementación, las pruebas y el análisis de rendimiento de un módulo de disparadores en PostgreSQL para el proyecto IoT Analytics Dashboard de NexoDev Solutions.

La idea central del módulo es que **el sistema reaccione solo a los eventos de la base de datos**. El consumidor que recibe las lecturas de los sensores únicamente hace un `INSERT` en la tabla `lecturas`; a partir de ese evento, los disparadores validan la lectura, generan alertas, actualizan el estado de cada sensor, registran en una **tabla de auditoría** los datos fuera de rango y cualquier cambio en los datos de los sensores, y **envían las alertas a RabbitMQ** para notificar las anomalías en tiempo real.

Repositorio del proyecto: https://github.com/luisgael-ml/iot-triggers

Competencia específica: manejar un lenguaje de programación visual y la conexión a la base de datos mediante código SQL embebido (clave de actividad BDM-1301T03CE01A05).

Objetivos específicos:

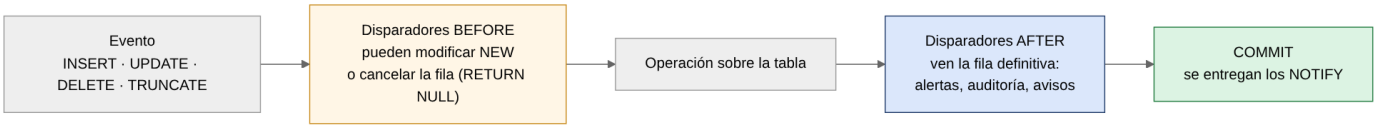
1. Diseñar el esquema de la base de datos en PostgreSQL con al menos tres tablas relacionadas con datos IoT.
2. Programar un disparador que registre automáticamente en una tabla de auditoría los cambios en los datos de los sensores.
3. Implementar un disparador que envíe alertas a través de RabbitMQ al identificar valores fuera de rango.
4. Probar los disparadores con datos simulados y documentar los resultados.
5. Presentar un informe técnico con el diseño del esquema, el código, las pruebas y el impacto en la gestión de datos IoT.

2. Marco teórico: disparadores

2.1 ¿Qué es un disparador?

Un disparador es código que la base de datos **ejecuta automáticamente cuando ocurre un evento** sobre una tabla: un `INSERT`, `UPDATE`, `DELETE` o `TRUNCATE`. A diferencia de una función, nadie lo llama de forma explícita: se ejecuta solo, dentro de la misma transacción que la operación que lo provocó. Si el disparador falla, se deshace todo, incluida la operación original.

En PostgreSQL un disparador tiene dos partes: una **función** que devuelve el tipo `trigger` y contiene la lógica, y la declaración **CREATE TRIGGER**, que indica la tabla, el evento, el momento y la granularidad. Esto permite reutilizar una misma función en varios disparadores, como la función de auditoría de este proyecto, que se usa en tres tablas.



Momento en que se ejecutan los disparadores dentro de una operación.

2.2 Clasificación

CRITERIO	OPCIONES	USO TÍPICO	EN EL PROYECTO
Momento	BEFORE	Validar o modificar la fila antes de guardarla; puede cancelarla con RETURN NULL	T1 valida lecturas; T7 llena campos de control; T9 bloquea la auditoría
	AFTER	Reaccionar con la fila ya guardada: alertas, auditoría, avisos	T2, T3, T5, T6 y T8
	INSTEAD OF	Solo en vistas: define qué hacer en lugar de la operación	No se usa
Granularidad	FOR EACH ROW	Se ejecuta una vez por cada fila afectada	T1, T2, T3, T6, T7, T8 y T9
	FOR EACH STATEMENT	Se ejecuta una vez por sentencia, aunque afecte miles de filas	T4 y T5
Evento	INSERT, UPDATE [OF columna], DELETE, TRUNCATE	Se pueden combinar con OR	T6: INSERT OR UPDATE OR DELETE; T8: UPDATE OF estado
Condición	WHEN (expresión)	Solo se ejecuta si se cumple la condición; ahorra llamadas	T8: solo cuando un equipo deja de estar activo

2.3 Variables especiales

VARIABLE	CONTENIDO	EJEMPLO EN EL PROYECTO
NEW	La fila nueva (INSERT y UPDATE). En BEFORE se puede modificar	T7 asigna `NEW.actualizado_en := NOW()`
OLD	La fila anterior (UPDATE y DELETE)	T6 compara OLD contra NEW para obtener solo las columnas que cambiaron
TG_OP	La operación: INSERT, UPDATE, DELETE o TRUNCATE	T6 decide qué datos guardar; T9 lo incluye en el mensaje de error
TG_TABLE_NAME	La tabla que disparó el evento	T6 la guarda en la columna `tabla` de la auditoría
Tablas de transición	`REFERENCING NEW TABLE AS nuevas`: todas las filas afectadas por la sentencia	T5 procesa en una sola consulta todas las lecturas insertadas

2.4 Valor de retorno

DISPARADOR	RETURN NEW	RETURN NULL
BEFORE por fila	Continúa con la fila (posiblemente modificada)	Cancela la operación solo para esa fila, sin error
AFTER o por sentencia	Se ignora	Se ignora (es la convención)
Para abortar todo	`RAISE EXCEPTION`: deshace la operación y la transacción completa	

2.5 Escenarios de aplicación

ESCENARIO	EJEMPLO GENERAL	EN EL IOT ANALYTICS DASHBOARD
Auditoría	Registrar quién cambió un salario y cuándo	T6 registra cambios de umbrales, estados y correcciones de lecturas
Validación compleja	Rechazar un pedido si el cliente tiene	T1 rechaza valores imposibles y lecturas de equipos

ESCENARIO	EJEMPLO GENERAL	EN EL IOT ANALYTICS DASHBOARD
	adeudos	fuera de servicio
Campos derivados	Actualizar el total de una factura	T5 mantiene el último valor y el total de lecturas de cada sensor
Integridad entre tablas	Descontar inventario al registrar una venta	T8 cierra las alertas pendientes al poner un equipo en mantenimiento
Notificaciones	Avisar a otro sistema de un evento	T3 y T4 envían las alertas a RabbitMQ
Protección de datos	Impedir borrar registros históricos	T9 hace inmutable la bitácora de auditoría

2.6 Ventajas, desventajas y buenas prácticas

VENTAJAS	DESVENTAJAS
Se ejecutan siempre, sin importar qué aplicación o usuario modifique los datos	Son “invisibles”: quien hace un INSERT no ve todo lo que ocurre detrás
Corren en la misma transacción: la operación y sus efectos se guardan o se deshacen juntos	Cada disparador por fila agrega costo a cada fila insertada
Centralizan reglas que de otro modo se repetirían en cada cliente	Disparadores encadenados son difíciles de depurar
La auditoría no depende de que la aplicación se acuerde de registrarla	No pueden comunicarse directamente con sistemas externos como RabbitMQ

Buenas prácticas aplicadas: lógica corta en cada disparador; preferir disparadores por sentencia cuando se procesan muchas filas; usar `WHEN` para no ejecutar la función sin necesidad; nombrarlos con un prefijo que indique el orden (PostgreSQL ejecuta los disparadores del mismo evento en orden alfabético); usar `SECURITY DEFINER` con `search_path` fijo; y nunca hacer operaciones lentas o externas dentro del disparador.

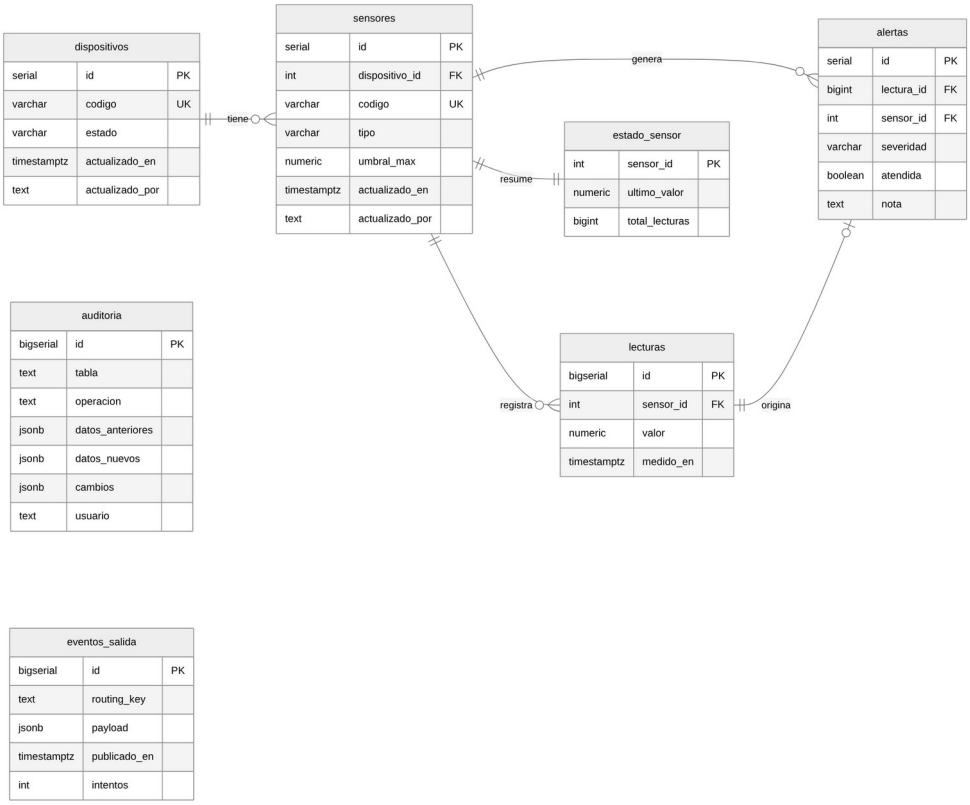
2.7 Disparadores y sistemas externos: el patrón outbox

Un disparador no puede conectarse a RabbitMQ, y aunque pudiera, sería un error: si publicara la alerta y después la transacción se deshiciera, RabbitMQ anunciaría una alerta que no existe; y si RabbitMQ estuviera caído, el INSERT de la lectura fallaría o se haría lento.

La solución es el **patrón outbox (bandeja de salida)**: el disparador solo inserta el evento en la tabla `eventos_salida`, dentro de la misma transacción, y avisa con `pg_notify`. PostgreSQL entrega ese aviso **solo cuando la transacción se confirma**. Un servicio puente escucha el aviso con `LISTEN`, publica los eventos pendientes en RabbitMQ y los marca como publicados. Si el puente o RabbitMQ fallan, los eventos esperan en la tabla y se publican al recuperarse: ninguna alerta se pierde ni se inventa.

3. Diseño del esquema

El esquema tiene siete tablas relacionadas. Las cuatro del proyecto (dispositivos, sensores, lecturas y alertas) se ampliaron con columnas de control (`actualizado_en`, `actualizado_por`, `nota`), y se agregaron tres tablas que alimentan los disparadores.



Esquema de la base de datos. auditoria y eventos_salida no tienen llaves foráneas para conservar el historial aunque se borren los registros originales.

TABLA	PROPÓSITO	QUIÉN LA LLENA
dispositivos, sensores	Catálogo de equipos y sensores con su umbral	Administración; T7 llena los campos de control
lecturas	Mediciones de los sensores	Consumidor (solo INSERT)
alertas	Lecturas que superaron el umbral	T2; T8 las cierra
estado_sensor	Último valor y total de lecturas por sensor	T5
auditoria	Bitácora inmutable de cambios y rechazos, en JSONB	T1 y T6
eventos_salida	Bandeja de salida hacia RabbitMQ	T1, T3 y T8; el puente la marca

4. Diseño de los disparadores

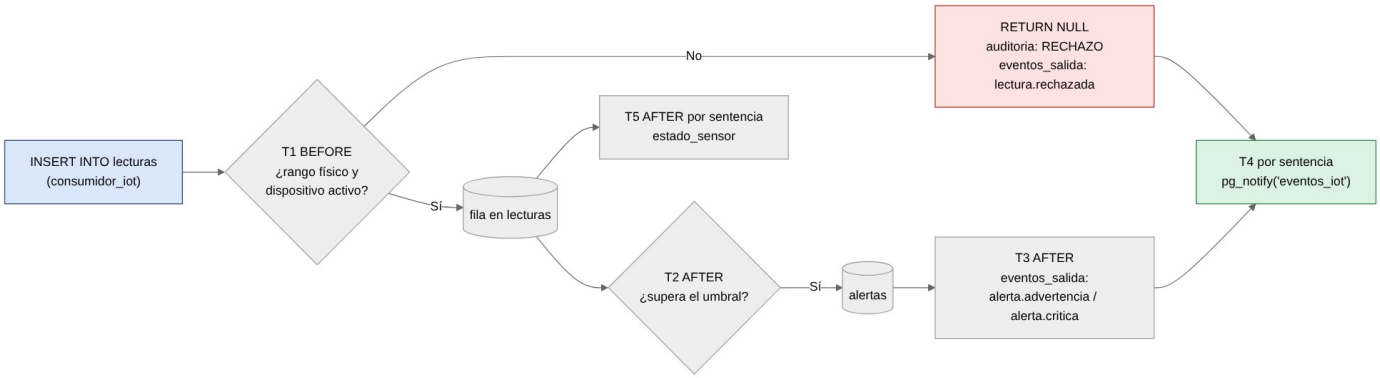
4.1 Catálogo

NO.	DISPARADOR	TABLA Y EVENTO	TIPO	QUÉ AUTOMATIZA
T1	`t1_lecturas_validar`	lecturas · INSERT	BEFORE · fila	Descarta lecturas fuera del rango físico o de equipos no activos, las audita y las encola
T2	`t2_lecturas_alerta`	lecturas · INSERT	AFTER · fila	Crea la alerta de advertencia o crítica
T3	`t3_alertas_notificar`	alertas · INSERT	AFTER · fila	Encola la alerta para RabbitMQ con su routing key
T4	`t4_eventos_avisar`	eventos_salida · INSERT	AFTER · sentencia	`pg_notify` para despertar al puente

NO.	DISPARADOR	TABLA Y EVENTO	TIPO	QUÉ AUTOMATIZA
T5	`t5_lecturas_estado`	lecturas · INSERT	AFTER · sentencia con tabla de transición	Actualiza estado_sensor
T6	`t6_auditar_*`	sensores, dispositivos · I/U/D; lecturas · U/D	AFTER · fila	Auditoría genérica con datos antes/después y columnas cambiadas
T7	`t7_*_marcar`	sensores, dispositivos · UPDATE	BEFORE · fila	Llena actualizado_en y actualizado_por
T8	`t8_dispositivo_fuera_servicio`	dispositivos · UPDATE OF estado	AFTER · fila · WHEN	Cierra alertas pendientes y avisa el cambio de estado
T9	`t9_auditoria_*`	auditoria · UPDATE, DELETE, TRUNCATE	BEFORE · fila y sentencia	Bloquea cualquier modificación de la bitácora

4.2 Flujo al insertar una lectura

Con un solo `INSERT` del consumidor se encadenan hasta cinco disparadores. El orden está garantizado: primero todos los BEFORE, después la escritura, luego los AFTER por fila y al final los AFTER por sentencia; dentro de cada grupo, en orden alfabético del nombre.

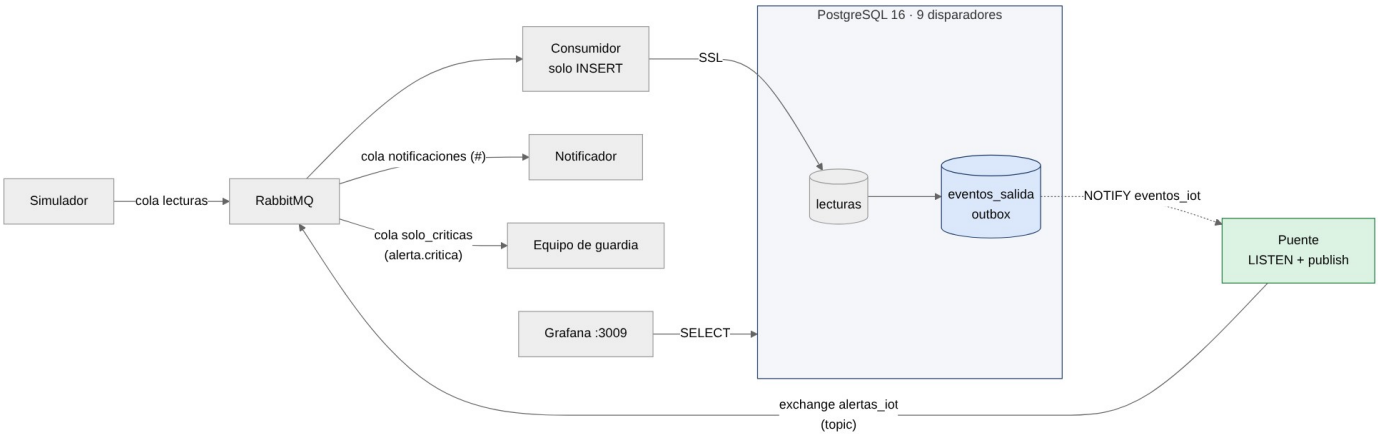


Disparadores que se encadenan con el INSERT de una lectura.

4.3 Auditoría de los cambios en los datos de los sensores

La función `trg_auditar` es una sola para todas las tablas. Convierte OLD y NEW a JSONB, los compara columna por columna y guarda solo las diferencias en el formato `{"columna": {"antes": ..., "despues": ...}}`. Si un UPDATE no cambió nada real, no registra nada. Como las funciones se ejecutan con `SECURITY DEFINER`, el usuario se toma de `session_user`, es decir, quien realmente inició la sesión y no el dueño de la función.

4.4 Alertas a RabbitMQ



Arquitectura: de la lectura a la notificación pasando por la bandeja de salida.

ROUTING KEY	ORIGEN	COLAS QUE LA RECIBEN
`alerta.advertencia`	T3 (valor mayor al umbral)	notificaciones
`alerta.critica`	T3 (valor mayor al umbral + 20 %)	notificaciones y solo_criticas
`lectura.rechazada`	T1 (valor imposible o equipo fuera de servicio)	notificaciones
`dispositivo.mantenimiento` / `dispositivo.inactivo`	T8	notificaciones

El exchange `alertas_iot` es de tipo topic: cada cola elige qué recibir con un patrón. La cola `notificaciones` usa `#` (todo) y la cola `solo_criticas` usa `alerta.critica`, por ejemplo para el equipo de guardia. El puente usa confirmaciones de publicación de RabbitMQ y `FOR UPDATE SKIP LOCKED`, para que varios puentes puedan trabajar en paralelo sin publicar dos veces el mismo evento.

4.5 Seguridad

USUARIO	PERMISOS	LO DEMÁS LO HACEN
`consumidor_iot`	INSERT en lecturas	T1, T2, T3, T4 y T5 con SECURITY DEFINER
`puente_rmq`	SELECT y UPDATE (publicado_en, intentos) en eventos_salida	
`operador_planta`	UPDATE de umbral, estado y valor	T6, T7 y T8 registran y reaccionan
`grafana_lector`	Solo SELECT	

5. Implementación

5.1 Estructura del proyecto

```
iot-triggers/
├── docker-compose.yml        → 7 contenedores en 2 redes
├── datos/
│   ├── init.sql              → 7 tablas, 9 disparadores, usuarios y datos de prueba
│   └── pruebas.sql           → pruebas de funcionamiento y de rendimiento
├── servicios/
│   ├── consumidor.py         → RabbitMQ → INSERT en lecturas (SQL embebido)
│   ├── puente.py             → LISTEN eventos_iot → publica en el exchange alertas_iot
│   ├── notificador.py        → suscriptor de las alertas
```

```
│   └── simulador.py          → 9 sensores IoT
├── grafana/                  → tablero de los disparadores
└── docs/Documentacion_Tarea9.pdf
```

5.2 Esquema y disparadores: init.sql

```
-- ======================================================================
-- TAREA 9 · Programacion del lado del servidor: disparadores (triggers)
-- IoT Analytics Dashboard (NexoDev Solutions) · PostgreSQL 16
-- El consumidor solo hace INSERT en lecturas; los disparadores automatizan
-- el resto: validar, alertar, auditar, actualizar campos y avisar a
-- RabbitMQ (patron outbox + LISTEN/NOTIFY).
-- ======================================================================

-- ============================== TABLAS ==============================
CREATE TABLE dispositivos (
    id             SERIAL PRIMARY KEY,
    codigo         VARCHAR(20) NOT NULL UNIQUE,
    nombre         VARCHAR(60) NOT NULL,
    ubicacion      VARCHAR(60) NOT NULL,
    estado         VARCHAR(15) NOT NULL DEFAULT 'activo'
                   CHECK (estado IN ('activo', 'mantenimiento', 'inactivo')),
    actualizado_en  TIMESTAMPTZ,
    actualizado_por TEXT
);

CREATE TABLE sensores (
    id              SERIAL PRIMARY KEY,
    dispositivo_id  INT NOT NULL REFERENCES dispositivos(id),
    codigo          VARCHAR(20) NOT NULL UNIQUE,
    tipo            VARCHAR(20) NOT NULL CHECK (tipo IN ('temperatura', 'vibracion', 'energia')),
    unidad          VARCHAR(10) NOT NULL,
    umbral_max      NUMERIC(10,2) NOT NULL CHECK (umbral_max > 0),
    actualizado_en  TIMESTAMPTZ,
    actualizado_por TEXT
);

CREATE TABLE lecturas (
    id        BIGSERIAL PRIMARY KEY,
    sensor_id INT NOT NULL REFERENCES sensores(id),
    valor     NUMERIC(10,2) NOT NULL,
    medido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_lecturas_sensor_fecha ON lecturas (sensor_id, medido_en DESC);
CREATE INDEX idx_lecturas_fecha        ON lecturas (medido_en DESC);

CREATE TABLE alertas (
    id         SERIAL PRIMARY KEY,
    lectura_id BIGINT NOT NULL REFERENCES lecturas(id) ON DELETE CASCADE,
    sensor_id  INT NOT NULL REFERENCES sensores(id),
    severidad  VARCHAR(12) NOT NULL CHECK (severidad IN ('advertencia', 'critica')),
    valor      NUMERIC(10,2) NOT NULL,
    umbral     NUMERIC(10,2) NOT NULL,
    creada_en  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    atendida   BOOLEAN NOT NULL DEFAULT FALSE,
    nota       TEXT
);
CREATE INDEX idx_alertas_pendientes ON alertas (sensor_id) WHERE NOT atendida;

-- Bitacora de auditoria: solo se inserta, nunca se modifica ni se borra
CREATE TABLE auditoria (
    id               BIGSERIAL PRIMARY KEY,
    tabla            TEXT NOT NULL,
    operacion        TEXT NOT NULL CHECK (operacion IN ('INSERT', 'UPDATE', 'DELETE', 'RECHAZO')),
    registro_id      TEXT,
    datos_anteriores JSONB,
    datos_nuevos     JSONB,
```

```
    cambios          JSONB,
    detalle          TEXT,
    usuario          TEXT NOT NULL DEFAULT session_user,
    ocurrido_en      TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX idx_auditoria_fecha ON auditoria (ocurrido_en DESC);

-- Bandeja de salida (outbox): eventos que el puente publica en RabbitMQ
CREATE TABLE eventos_salida (
    id           BIGSERIAL PRIMARY KEY,
    routing_key  TEXT NOT NULL,
    payload      JSONB NOT NULL,
    creado_en    TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    publicado_en TIMESTAMPTZ,
    intentos     INT NOT NULL DEFAULT 0
);
CREATE INDEX idx_eventos_pendientes ON eventos_salida (id) WHERE publicado_en IS NULL;

-- Estado en vivo de cada sensor, lo mantiene un disparador por sentencia
CREATE TABLE estado_sensor (
    sensor_id      INT PRIMARY KEY REFERENCES sensores(id),
    ultima_lectura TIMESTAMPTZ NOT NULL,
    ultimo_valor   NUMERIC(10,2) NOT NULL,
    total_lecturas BIGINT NOT NULL,
    actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ========================= FUNCIONES AUXILIARES =========================
CREATE FUNCTION fn_clasificar_lectura(p_valor NUMERIC, p_umbral NUMERIC)
RETURNS TEXT LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE WHEN p_valor > p_umbral * 1.2 THEN 'critica'
                WHEN p_valor > p_umbral       THEN 'advertencia'
                ELSE 'normal' END;
$$;

CREATE FUNCTION fn_valor_valido(p_tipo TEXT, p_valor NUMERIC)
RETURNS BOOLEAN LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE p_tipo
        WHEN 'temperatura' THEN p_valor BETWEEN -40 AND 150
        WHEN 'vibracion'   THEN p_valor BETWEEN 0 AND 50
        WHEN 'energia'     THEN p_valor BETWEEN 0 AND 1000
        ELSE FALSE END;
$$;

-- ============================ DISPARADORES ============================

-- T1. BEFORE INSERT en lecturas: validar la lectura antes de guardarla.
-- Si el valor esta fuera del rango fisico o el dispositivo no esta activo,
-- registra el rechazo en auditoria, encola un evento para RabbitMQ y
-- devuelve NULL: la fila no se inserta, pero la transaccion continua.
CREATE FUNCTION trg_lecturas_validar() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_sensor  RECORD;
    v_motivo  TEXT;
BEGIN
    SELECT s.codigo, s.tipo, s.unidad, d.codigo AS dispositivo, d.estado
      INTO v_sensor
      FROM sensores s JOIN dispositivos d ON d.id = s.dispositivo_id
     WHERE s.id = NEW.sensor_id;

    IF v_sensor.estado <> 'activo' THEN
        v_motivo := format('dispositivo %s en %s', v_sensor.dispositivo, v_sensor.estado);
    ELSIF NOT fn_valor_valido(v_sensor.tipo, NEW.valor) THEN
        v_motivo := format('valor %s fuera del rango fisico de %s', NEW.valor, v_sensor.tipo);
    END IF;

    IF v_motivo IS NOT NULL THEN
```

```
        INSERT INTO auditoria (tabla, operacion, datos_nuevos, detalle)
        VALUES ('lecturas', 'RECHAZO', to_jsonb(NEW) || jsonb_build_object('sensor',
v_sensor.codigo), v_motivo);

        INSERT INTO eventos_salida (routing_key, payload)
        VALUES ('lectura.rechazada', jsonb_build_object(
                'sensor', v_sensor.codigo, 'valor', NEW.valor, 'unidad', v_sensor.unidad,
                'motivo', v_motivo, 'medido_en', NEW.medido_en));
        RETURN NULL;                                    -- descarta la fila
    END IF;
    RETURN NEW;
END $$;

CREATE TRIGGER t1_lecturas_validar
BEFORE INSERT ON lecturas
FOR EACH ROW EXECUTE FUNCTION trg_lecturas_validar();

-- T2. AFTER INSERT en lecturas: generar la alerta si supera el umbral.
CREATE FUNCTION trg_lecturas_alerta() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_umbral NUMERIC;
    v_sev    TEXT;
BEGIN
    SELECT umbral_max INTO v_umbral FROM sensores WHERE id = NEW.sensor_id;
    v_sev := fn_clasificar_lectura(NEW.valor, v_umbral);
    IF v_sev <> 'normal' THEN
        INSERT INTO alertas (lectura_id, sensor_id, severidad, valor, umbral, creada_en)
        VALUES (NEW.id, NEW.sensor_id, v_sev, NEW.valor, v_umbral, NEW.medido_en);
    END IF;
    RETURN NULL;                                        -- en AFTER se ignora
END $$;

CREATE TRIGGER t2_lecturas_alerta
AFTER INSERT ON lecturas
FOR EACH ROW EXECUTE FUNCTION trg_lecturas_alerta();

-- T3. AFTER INSERT en alertas: encolar la notificacion para RabbitMQ.
-- La routing key indica la severidad: alerta.advertencia / alerta.critica.
CREATE FUNCTION trg_alertas_notificar() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    INSERT INTO eventos_salida (routing_key, payload)
    SELECT 'alerta.' || NEW.severidad,
           jsonb_build_object('alerta_id', NEW.id, 'sensor', s.codigo, 'dispositivo', d.codigo,
                              'valor', NEW.valor, 'umbral', NEW.umbral, 'unidad', s.unidad,
                              'severidad', NEW.severidad, 'medido_en', NEW.creada_en)
    FROM sensores s JOIN dispositivos d ON d.id = s.dispositivo_id
    WHERE s.id = NEW.sensor_id;
    RETURN NULL;
END $$;

CREATE TRIGGER t3_alertas_notificar
AFTER INSERT ON alertas
FOR EACH ROW EXECUTE FUNCTION trg_alertas_notificar();

-- T4. AFTER INSERT en eventos_salida, POR SENTENCIA: despertar al puente.
-- pg_notify se entrega al confirmar la transaccion; una sola vez por sentencia.
CREATE FUNCTION trg_eventos_avisar() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    PERFORM pg_notify('eventos_iot', 'nuevos');
    RETURN NULL;
END $$;

CREATE TRIGGER t4_eventos_avisar
AFTER INSERT ON eventos_salida
FOR EACH STATEMENT EXECUTE FUNCTION trg_eventos_avisar();
```

```
-- T5. AFTER INSERT en lecturas, POR SENTENCIA con tabla de transicion:
-- actualiza estado_sensor una vez por sentencia, aunque se inserten miles de filas.
CREATE FUNCTION trg_lecturas_estado() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    INSERT INTO estado_sensor AS e (sensor_id, ultima_lectura, ultimo_valor, total_lecturas)
    SELECT DISTINCT ON (sensor_id)
           sensor_id, medido_en, valor, COUNT(*) OVER (PARTITION BY sensor_id)
    FROM nuevas
    ORDER BY sensor_id, medido_en DESC
    ON CONFLICT (sensor_id) DO UPDATE SET
        ultimo_valor   = CASE WHEN EXCLUDED.ultima_lectura >= e.ultima_lectura
                              THEN EXCLUDED.ultimo_valor ELSE e.ultimo_valor END,
        ultima_lectura = GREATEST(e.ultima_lectura, EXCLUDED.ultima_lectura),
        total_lecturas = e.total_lecturas + EXCLUDED.total_lecturas,
        actualizado_en = NOW();
    RETURN NULL;
END $$;

CREATE TRIGGER t5_lecturas_estado
AFTER INSERT ON lecturas
REFERENCING NEW TABLE AS nuevas
FOR EACH STATEMENT EXECUTE FUNCTION trg_lecturas_estado();

-- T6. Auditoria generica (una sola funcion para varias tablas).
-- Guarda los datos antes y despues, solo las columnas que cambiaron y el
-- usuario real de la sesion (session_user, no el dueno de la funcion).
CREATE FUNCTION trg_auditar() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_viejo   JSONB := CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN to_jsonb(OLD) END;
    v_nuevo   JSONB := CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN to_jsonb(NEW) END;
    v_cambios JSONB;
BEGIN
    IF TG_OP = 'UPDATE' THEN
        SELECT jsonb_object_agg(n.key, jsonb_build_object('antes', o.value, 'despues', n.value))
          INTO v_cambios
          FROM jsonb_each(v_nuevo) n JOIN jsonb_each(v_viejo) o USING (key)
         WHERE n.value IS DISTINCT FROM o.value
           AND n.key NOT IN ('actualizado_en', 'actualizado_por');
        IF v_cambios IS NULL THEN RETURN NULL; END IF;   -- UPDATE sin cambios reales
    END IF;

    INSERT INTO auditoria (tabla, operacion, registro_id, datos_anteriores, datos_nuevos, cambios)
    VALUES (TG_TABLE_NAME, TG_OP, COALESCE(v_nuevo, v_viejo) ->> 'id', v_viejo, v_nuevo,
v_cambios);
    RETURN NULL;
END $$;

CREATE TRIGGER t6_auditar_sensores
AFTER INSERT OR UPDATE OR DELETE ON sensores
FOR EACH ROW EXECUTE FUNCTION trg_auditar();

CREATE TRIGGER t6_auditar_dispositivos
AFTER INSERT OR UPDATE OR DELETE ON dispositivos
FOR EACH ROW EXECUTE FUNCTION trg_auditar();

-- En lecturas solo se auditan correcciones y borrados (los INSERT son el flujo normal)
CREATE TRIGGER t6_auditar_lecturas
AFTER UPDATE OR DELETE ON lecturas
FOR EACH ROW EXECUTE FUNCTION trg_auditar();

-- T7. BEFORE UPDATE: actualizacion automatica de campos de control.
CREATE FUNCTION trg_marcar_actualizacion() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.actualizado_en  := NOW();
```

```
    NEW.actualizado_por := session_user;
    RETURN NEW;                                          -- devuelve la fila modificada
END $$;

CREATE TRIGGER t7_sensores_marcar
BEFORE UPDATE ON sensores
FOR EACH ROW EXECUTE FUNCTION trg_marcar_actualizacion();

CREATE TRIGGER t7_dispositivos_marcar
BEFORE UPDATE ON dispositivos
FOR EACH ROW EXECUTE FUNCTION trg_marcar_actualizacion();

-- T8. AFTER UPDATE OF estado en dispositivos, con condicion WHEN:
-- al pasar a mantenimiento o inactivo, cierra sus alertas pendientes y avisa.
CREATE FUNCTION trg_dispositivo_fuera_servicio() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_cerradas INT;
BEGIN
    UPDATE alertas a SET atendida = TRUE,
                         nota = format('Cerrada automaticamente: %s paso a %s', NEW.codigo,
NEW.estado)
      FROM sensores s
     WHERE s.id = a.sensor_id AND s.dispositivo_id = NEW.id AND NOT a.atendida;
    GET DIAGNOSTICS v_cerradas = ROW_COUNT;

    INSERT INTO eventos_salida (routing_key, payload)
    VALUES ('dispositivo.' || NEW.estado, jsonb_build_object(
            'dispositivo', NEW.codigo, 'estado_anterior', OLD.estado, 'estado', NEW.estado,
            'alertas_cerradas', v_cerradas, 'usuario', session_user));
    RETURN NULL;
END $$;

CREATE TRIGGER t8_dispositivo_fuera_servicio
AFTER UPDATE OF estado ON dispositivos
FOR EACH ROW
WHEN (OLD.estado = 'activo' AND NEW.estado <> 'activo')
EXECUTE FUNCTION trg_dispositivo_fuera_servicio();

-- T9. BEFORE UPDATE, DELETE o TRUNCATE en auditoria: la bitacora es inmutable.
CREATE FUNCTION trg_auditoria_inmutable() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'La auditoria es de solo insercion: no se permite % ', TG_OP
          USING ERRCODE = 'insufficient_privilege';
END $$;

CREATE TRIGGER t9_auditoria_inmutable
BEFORE UPDATE OR DELETE ON auditoria
FOR EACH ROW EXECUTE FUNCTION trg_auditoria_inmutable();

-- TRUNCATE no dispara triggers por fila: se protege con uno por sentencia
CREATE TRIGGER t9_auditoria_sin_truncate
BEFORE TRUNCATE ON auditoria
FOR EACH STATEMENT EXECUTE FUNCTION trg_auditoria_inmutable();

-- ======================= USUARIOS Y PERMISOS =======================
-- Consumidor: solo inserta lecturas; todo lo demas lo hacen los disparadores
CREATE ROLE consumidor_iot LOGIN PASSWORD 'consumidor123';
GRANT SELECT (id, codigo) ON sensores TO consumidor_iot;
GRANT INSERT ON lecturas TO consumidor_iot;
GRANT USAGE ON SEQUENCE lecturas_id_seq TO consumidor_iot;

-- Puente hacia RabbitMQ: solo lee y marca la bandeja de salida
CREATE ROLE puente_rmq LOGIN PASSWORD 'puente123';
GRANT SELECT, UPDATE (publicado_en, intentos) ON eventos_salida TO puente_rmq;

-- Operador: cambia umbrales y estados (para probar la auditoria)
```

```
CREATE ROLE operador_planta LOGIN PASSWORD 'operador123';
GRANT SELECT ON sensores, dispositivos, lecturas TO operador_planta;
GRANT UPDATE (umbral_max) ON sensores TO operador_planta;
GRANT UPDATE (estado) ON dispositivos TO operador_planta;
GRANT UPDATE (valor) ON lecturas TO operador_planta;

-- Grafana: solo lectura
CREATE ROLE grafana_lector LOGIN PASSWORD 'grafana123';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO grafana_lector;

-- ========================== DATOS INICIALES ==========================
INSERT INTO dispositivos (codigo, nombre, ubicacion) VALUES
('PRE-03', 'Prensa hidraulica 3', 'Linea 1'),
('TOR-01', 'Torno CNC 1',         'Linea 2'),
('EMB-02', 'Embotelladora 2',     'Linea 3');

INSERT INTO sensores (dispositivo_id, codigo, tipo, unidad, umbral_max) VALUES
(1, 'PRE03-TEMP', 'temperatura', '°C',   85.00),
(1, 'PRE03-VIB',  'vibracion',   'mm/s',  7.00),
(1, 'PRE03-ENE',  'energia',     'kWh',  30.00),
(2, 'TOR01-TEMP', 'temperatura', '°C',   80.00),
(2, 'TOR01-VIB',  'vibracion',   'mm/s',  6.00),
(2, 'TOR01-ENE',  'energia',     'kWh',  25.00),
(3, 'EMB02-TEMP', 'temperatura', '°C',   75.00),
(3, 'EMB02-VIB',  'vibracion',   'mm/s',  5.00),
(3, 'EMB02-ENE',  'energia',     'kWh',  20.00);

-- Historico de 1 dia (una lectura cada 30 s por sensor). Se carga con los
-- disparadores de usuario desactivados para no publicar miles de alertas
-- antiguas en RabbitMQ; despues se calcula el estado una sola vez.
ALTER TABLE lecturas DISABLE TRIGGER USER;
SELECT setseed(0.9);
INSERT INTO lecturas (sensor_id, valor, medido_en)
SELECT s.id, ROUND((s.umbral_max * (0.80 + (random() - 0.5) * 0.08))::numeric, 2), t
FROM sensores s
CROSS JOIN generate_series(NOW() - INTERVAL '1 day', NOW() - INTERVAL '1 minute',
                           INTERVAL '30 seconds') AS t;
ALTER TABLE lecturas ENABLE TRIGGER USER;

INSERT INTO estado_sensor (sensor_id, ultima_lectura, ultimo_valor, total_lecturas)
SELECT DISTINCT ON (sensor_id) sensor_id, medido_en, valor, COUNT(*) OVER (PARTITION BY sensor_id)
FROM lecturas ORDER BY sensor_id, medido_en DESC;
ANALYZE;
```

5.3 Consumidor: solo un INSERT

El consumidor no conoce las reglas de negocio. Si el disparador T1 descarta la lectura, el INSERT devuelve 0 filas y el consumidor solo lo registra.

```
"""
Consumidor: toma las lecturas de RabbitMQ y solo hace un INSERT en la tabla
lecturas con el usuario consumidor_iot. No valida ni genera alertas: de eso
se encargan los disparadores de la base de datos.
"""
import json
import os
import time

import pika
import psycopg

CONNINFO = (f"host={os.getenv('DB_HOST', 'datos')} dbname=iotanalytics "
            f"user={os.getenv('DB_USER', 'consumidor_iot')} "
            f"password={os.environ['DB_PASSWORD']} sslmode=require")

INSERTAR = """INSERT INTO lecturas (sensor_id, valor, medido_en)
```

```
            SELECT id, %s::numeric, %s::timestamptz FROM sensores WHERE codigo = %s"""


def conectar_bd():
    while True:
        try:
            return psycopg.connect(CONNINFO, autocommit=True)
        except psycopg.OperationalError:
            print("PostgreSQL no disponible, reintentando...")
            time.sleep(3)


def conectar_rabbit():
    params = pika.URLParameters(os.getenv("RABBIT_URL", "amqp://iot:iot123@mensajes:5672/"))
    while True:
        try:
            return pika.BlockingConnection(params)
        except pika.exceptions.AMQPConnectionError:
            print("RabbitMQ no disponible, reintentando...")
            time.sleep(3)


def main():
    bd = conectar_bd()
    print("Conectado a PostgreSQL con SSL:", bd.info.pgconn.ssl_in_use)
    con = conectar_rabbit()
    canal = con.channel()
    canal.queue_declare(queue="lecturas", durable=True)
    canal.basic_qos(prefetch_count=50)
    cuenta = {"insertadas": 0, "descartadas": 0}

    def al_recibir(ch, metodo, _props, cuerpo):
        lectura = json.loads(cuerpo)
        filas = bd.execute(INSERTAR, (lectura["valor"], lectura["medido_en"],
lectura["sensor"])).rowcount
        if filas:
            cuenta["insertadas"] += 1
        else:                                   # el disparador T1 devolvio NULL
            cuenta["descartadas"] += 1
            print(f"Descartada por t1_lecturas_validar: {lectura['sensor']} = {lectura['valor']}")
        ch.basic_ack(delivery_tag=metodo.delivery_tag)

    def reporte():
        print(f"{cuenta['insertadas']} insertadas · {cuenta['descartadas']} descartadas por
disparador")
        con.call_later(30, reporte)

    con.call_later(30, reporte)
    canal.basic_consume(queue="lecturas", on_message_callback=al_recibir)
    print("Consumidor escuchando la cola 'lecturas'")
    canal.start_consuming()


if __name__ == "__main__":
    main()
```

5.4 Puente PostgreSQL → RabbitMQ

```
"""
Puente PostgreSQL -> RabbitMQ (patron outbox).
Los disparadores no pueden hablar con RabbitMQ: dejan cada evento en la tabla
eventos_salida y avisan con pg_notify('eventos_iot'). Este servicio escucha ese
aviso (LISTEN), publica los eventos pendientes en el exchange 'alertas_iot' y
los marca como publicados. Si el puente se detiene, los eventos esperan en la
tabla y se publican al volver: ninguna alerta se pierde.
"""
import json
```

```
import os
import time

import pika
import psycopg

CONNINFO = (f"host={os.getenv('DB_HOST', 'datos')} dbname=iotanalytics "
            f"user={os.getenv('DB_USER', 'puente_rmq')} "
            f"password={os.environ['DB_PASSWORD']} sslmode=require")
EXCHANGE = "alertas_iot"

PENDIENTES = """SELECT id, routing_key, payload FROM eventos_salida
                WHERE publicado_en IS NULL ORDER BY id
                LIMIT 100 FOR UPDATE SKIP LOCKED"""


def conectar_bd():
    while True:
        try:
            return psycopg.connect(CONNINFO, autocommit=True)
        except psycopg.OperationalError:
            print("PostgreSQL no disponible, reintentando...")
            time.sleep(3)


def conectar_rabbit():
    params = pika.URLParameters(os.getenv("RABBIT_URL", "amqp://iot:iot123@mensajes:5672/"))
    while True:
        try:
            con = pika.BlockingConnection(params)
            canal = con.channel()
            canal.exchange_declare(exchange=EXCHANGE, exchange_type="topic", durable=True)
            canal.confirm_delivery()           # RabbitMQ confirma cada mensaje recibido
            return con, canal
        except pika.exceptions.AMQPConnectionError:
            print("RabbitMQ no disponible, reintentando...")
            time.sleep(3)


def publicar_pendientes(bd, canal):
    publicados = 0
    with bd.transaction():
        for id_, clave, payload in bd.execute(PENDIENTES).fetchall():
            try:
                canal.basic_publish(exchange=EXCHANGE, routing_key=clave,
                                    body=json.dumps(payload, default=str, ensure_ascii=False),
                                    properties=pika.BasicProperties(
                                        content_type="application/json", delivery_mode=2))
                bd.execute("UPDATE eventos_salida SET publicado_en = clock_timestamp(), "
                           "intentos = intentos + 1 WHERE id = %s", (id_,))
                publicados += 1
            except pika.exceptions.AMQPError as e:
                bd.execute("UPDATE eventos_salida SET intentos = intentos + 1 WHERE id = %s",
(id_,))
                print(f"No se pudo publicar el evento {id_}: {e}")
                break
    return publicados


def main():
    bd = conectar_bd()
    print("Conectado a PostgreSQL con SSL:", bd.info.pgconn.ssl_in_use)
    rmq, canal = conectar_rabbit()
    bd.execute("LISTEN eventos_iot")
    print("Puente escuchando el canal 'eventos_iot'")
    total = publicar_pendientes(bd, canal)     # lo que quedo pendiente mientras no corria
    while True:
        # espera un NOTIFY hasta 5 s; si no llega, revisa de todos modos
```

```
        for _ in bd.notifies(timeout=5, stop_after=1):
            pass
        n = publicar_pendientes(bd, canal)
        if n:
            total += n
            print(f"Publicados {n} eventos en '{EXCHANGE}' (total {total})")
        rmq.process_data_events(time_limit=0)  # mantiene viva la conexion (heartbeat)


if __name__ == "__main__":
    main()
```

5.5 Notificador

```
"""
Notificador: suscriptor del exchange 'alertas_iot'. Representa al equipo de
operacion que recibe las alertas en tiempo real (correo, Slack, celular...).
Cola 'notificaciones' -> todos los eventos (#)
Cola 'solo_criticas'  -> solo alerta.critica (muestra el ruteo por routing key)
"""
import json
import os
import time

import pika

EXCHANGE = "alertas_iot"
ICONOS = {"alerta.critica": "[CRITICA]", "alerta.advertencia": "[ADVERTENCIA]",
          "lectura.rechazada": "[RECHAZADA]"}


def conectar():
    params = pika.URLParameters(os.getenv("RABBIT_URL", "amqp://iot:iot123@mensajes:5672/"))
    while True:
        try:
            return pika.BlockingConnection(params)
        except pika.exceptions.AMQPConnectionError:
            print("RabbitMQ no disponible, reintentando...")
            time.sleep(3)


def main():
    con = conectar()
    canal = con.channel()
    canal.exchange_declare(exchange=EXCHANGE, exchange_type="topic", durable=True)
    canal.queue_declare(queue="notificaciones", durable=True)
    canal.queue_bind(queue="notificaciones", exchange=EXCHANGE, routing_key="#")
    canal.queue_declare(queue="solo_criticas", durable=True,
                        arguments={"x-max-length": 1000})
    canal.queue_bind(queue="solo_criticas", exchange=EXCHANGE, routing_key="alerta.critica")

    def al_recibir(ch, metodo, _props, cuerpo):
        e = json.loads(cuerpo)
        clave = metodo.routing_key
        if clave.startswith("alerta."):
            texto = f"{e['sensor']} ({e['dispositivo']}) = {e['valor']} {e['unidad']}, umbral
{e['umbral']}"
        elif clave == "lectura.rechazada":
            texto = f"{e['sensor']} = {e['valor']} -> {e['motivo']}"
        else:
            texto = (f"{e['dispositivo']}: {e['estado_anterior']} -> {e['estado']} "
                     f"por {e['usuario']}, {e['alertas_cerradas']} alertas cerradas")
        print(f"{ICONOS.get(clave, '[DISPOSITIVO]')} {texto}")
        ch.basic_ack(delivery_tag=metodo.delivery_tag)

    canal.basic_consume(queue="notificaciones", on_message_callback=al_recibir)
    print(f"Notificador suscrito a '{EXCHANGE}'")
```

```
    canal.start_consuming()


if __name__ == "__main__":
    main()
```

5.6 Orquestación con Docker Compose

```
# Tarea 9 · Disparadores (triggers) · IoT Analytics Dashboard (NexoDev Solutions)
name: iot-triggers

services:
  # ================= BASE DE DATOS (tablas + disparadores) =================
  datos:
    image: postgres:16
    container_name: triggers-datos
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD:-admin123}
      POSTGRES_DB: iotanalytics
    command: >
      -c ssl=on -c ssl_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
      -c ssl_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
    volumes:
      - ./datos/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks: [red-datos]          # sin puerto expuesto a la PC
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d iotanalytics"]
      interval: 3s
      retries: 30

  # ================= MENSAJERIA EN TIEMPO REAL =================
  mensajes:
    image: rabbitmq:3.13-management
    container_name: triggers-rabbitmq
    hostname: triggers-rabbitmq      # nombre fijo del nodo
    environment:
      RABBITMQ_DEFAULT_USER: iot
      RABBITMQ_DEFAULT_PASS: iot123
      # La cookie de Erlang va como argumento: RabbitMQ ya no lee
      # /var/lib/rabbitmq/.erlang.cookie, que falla con "eacces" en Docker Desktop
      RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS: "-setcookie iottriggerscookie"
    tmpfs:
      - /var/lib/rabbitmq:mode=1777
    ports:
      - "15679:15672"              # consola de RabbitMQ: http://localhost:15679
    networks: [red-mensajes]
    healthcheck:
      # Prueba que el puerto AMQP acepte conexiones (no usa la cookie)
      test: ["CMD-SHELL", "bash -c '</dev/tcp/localhost/5672'"]
      interval: 5s
      retries: 30

  # ========= CONSUMIDOR: RabbitMQ -> INSERT en lecturas =========
  consumidor:
    build: ./servicios
    container_name: triggers-consumidor
    command: python -u consumidor.py
    environment:
      DB_HOST: datos
      DB_USER: consumidor_iot        # solo puede insertar lecturas
      DB_PASSWORD: consumidor123
      RABBIT_URL: amqp://iot:iot123@mensajes:5672/
    networks: [red-datos, red-mensajes]
    depends_on:
      datos: { condition: service_healthy }
      mensajes: { condition: service_healthy }
```

```
  # ================= SIMULADOR DE SENSORES =================
  simulador:
    build: ./servicios
    container_name: triggers-simulador
    command: python -u simulador.py
    environment:
      RABBIT_URL: amqp://iot:iot123@mensajes:5672/
      INTERVALO: "1"
    networks: [red-mensajes]       # no tiene acceso a la base de datos
    depends_on:
      consumidor: { condition: service_started }

  # ========= PUENTE: eventos_salida (outbox) -> RabbitMQ =========
  puente:
    build: ./servicios
    container_name: triggers-puente
    command: python -u puente.py
    environment:
      DB_HOST: datos
      DB_USER: puente_rmq            # solo lee y marca eventos_salida
      DB_PASSWORD: puente123
      RABBIT_URL: amqp://iot:iot123@mensajes:5672/
    networks: [red-datos, red-mensajes]
    depends_on:
      datos: { condition: service_healthy }
      mensajes: { condition: service_healthy }

  # ========= NOTIFICADOR: suscriptor de las alertas =========
  notificador:
    build: ./servicios
    container_name: triggers-notificador
    command: python -u notificador.py
    environment:
      RABBIT_URL: amqp://iot:iot123@mensajes:5672/
    networks: [red-mensajes]       # no tiene acceso a la base de datos
    depends_on:
      puente: { condition: service_started }

  # ================= VISUALIZACION =================
  grafana:
    image: grafana/grafana:11.3.0
    container_name: triggers-grafana
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin123
      GF_AUTH_ANONYMOUS_ENABLED: "true"
      GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
      GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH: /var/lib/grafana/dashboards/iot-triggers.json
    ports:
      - "3009:3000"                # http://localhost:3009
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
    networks: [red-datos]
    depends_on:
      datos: { condition: service_healthy }

networks:
  red-datos:
  red-mensajes:
```

6. Pruebas con datos simulados

La base inicia con un día de histórico (25,911 lecturas), cargado con los disparadores de usuario desactivados para no publicar miles de alertas antiguas. Después, el simulador envía 9 lecturas por segundo con picos sobre el umbral y lecturas defectuosas (-999).

NO.	PRUEBA	RESULTADO ESPERADO	RESULTADO OBTENIDO
P1	Arranque	7 contenedores en ejecución	Correcto: datos y RabbitMQ healthy; 5 servicios iniciados
P2	Disparadores creados	9 disparadores en 6 tablas	Correcto: 13 declaraciones (T6, T7 y T9 se aplican a varias tablas o eventos)
P3	Lectura normal (T1, T2)	Se guarda sin alerta	Correcto: lectura 27,073 de 70.00 °C
P4	Lectura crítica (T2, T3)	Alerta crítica y evento encolado	Correcto: alerta 32 de 110.00 °C; evento alerta.critica publicado
P5	Valor imposible (T1)	INSERT 0 0, auditoría y evento	Correcto: “valor -999.00 fuera del rango físico de vibración”
P6	Estado en vivo (T5)	Último valor y total actualizados	Correcto: PRE03-TEMP 110.00, 3,010 lecturas
P7	Auditoría de cambios (T6, T7)	Antes, después y usuario real	Correcto: umbral 85 → 88 y estado activo → mantenimiento por operador_planta
P8	Bitácora inmutable (T9)	Rechazar DELETE	Correcto: “La auditoría es de solo inserción”
P9	Alertas en tiempo real (T3, T4, puente)	Eventos publicados sin pendientes	Correcto: 0 pendientes, latencia de 42 a 128 ms; 241 eventos publicados y recibidos por el notificador
P10	Rendimiento	Por sentencia más rápido que por fila	Correcto: 1,571 ms contra 3,331 ms (2.1 veces)

Evidencia 1 — P1: los siete contenedores en ejecución.



Arranque del entorno.

Evidencia 2 — P2: disparadores creados y la tabla a la que pertenece cada uno.



Disparadores en PostgreSQL.

Evidencia 3 — P3 a P6: lectura normal, lectura crítica con su alerta, lectura imposible descartada y auditada, bandeja de salida (las primeras ya publicadas por el puente) y estado en vivo de los sensores.

```
C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>docker exec -i triggers-datos psql -U postgres -d iotanalytics < datos/pruebas.sql
=== A. Lectura normal: T1 la acepta, T2 no genera alerta ===
  id   | valor
-------+-------
 27073 | 70.00

INSERT 0 1
=== B. Lectura critica: T2 crea la alerta y T3 la encola para RabbitMQ ===
  id   | valor
-------+-------
 27074 | 110.00

INSERT 0 1
 alerta |   sensor   | severidad | valor  | umbral
--------+------------+-----------+--------+--------
     32 | PRE03-TEMP | critica   | 110.00 |  85.00

=== C. Valor imposible: T1 la descarta (INSERT 0 0), la audita y la encola ===
INSERT 0 0
 id |  tabla   | operacion |  sensor   |                      detalle
----+----------+-----------+-----------+-------------------------------------------------
 18 | lecturas | RECHAZO   | EMB02-VIB | valor -999.00 fuera del rango fisico de vibracion

=== D. Bandeja de salida: eventos generados por los disparadores ===
 id |    routing_key     |   sensor   |  valor  | publicado
----+--------------------+------------+---------+-----------
 38 | lectura.rechazada  | EMB02-VIB  | -999.00 | f
 37 | alerta.critica     | PRE03-TEMP | 110.00  | t
 36 | alerta.advertencia | EMB02-TEMP | 82.47   | t

=== E. T5: estado_sensor actualizado por el disparador por sentencia ===
   codigo   | ultimo_valor | total_lecturas |   hora
------------+--------------+----------------+----------
 PRE03-TEMP |       110.00 |           3010 | 11:43:24
 PRE03-VIB  |         4.44 |           3007 | 11:43:24
 PRE03-ENE  |        26.15 |           3008 | 11:43:24

=== F. Rendimiento: 10,000 lecturas normales (sin alertas) en una sola sentencia ===
Timing is on.
```

Pruebas de funcionamiento de los disparadores.

Evidencia 4 — P7 y P8: el operador cambia un umbral y pone TOR-01 en mantenimiento; la auditoría guarda solo las columnas que cambiaron y el usuario real. Ni el administrador puede borrar la bitácora.

```
C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>docker exec triggers-datos psql -U operador_planta -d iotanalytics -c "UPDATE sensores SET umbral_max = 8
8 WHERE codigo = 'PRE03-TEMP'"
UPDATE 1

C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>docker exec triggers-datos psql -U operador_planta -d iotanalytics -c "UPDATE dispositivos SET estado = '
mantenimiento' WHERE codigo = 'TOR-01'"
UPDATE 1

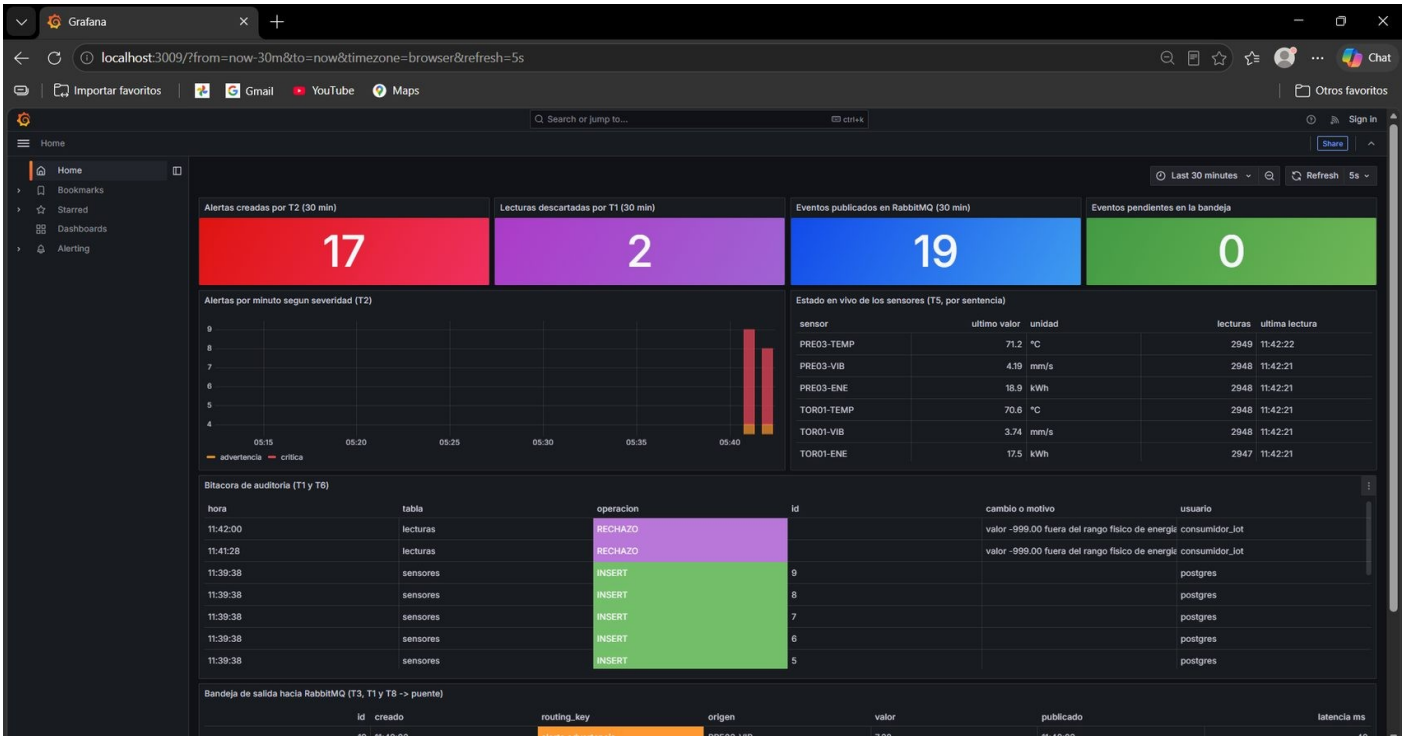
C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>docker exec triggers-datos psql -U postgres -d iotanalytics -c "SELECT tabla, operacion, cambios, usuario
 FROM auditoria WHERE operacion = 'UPDATE'"
    tabla     | operacion |                                cambios                                |     usuario
--------------+-----------+-----------------------------------------------------------------------+-----------------
 sensores     | UPDATE    | {"umbral_max": {"antes": 85.00, "despues": 88.00}}                    | operador_planta
 dispositivos | UPDATE    | {"estado": {"antes": "activo", "despues": "mantenimiento"}} | operador_planta
(2 rows)


C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>docker exec triggers-datos psql -U postgres -d iotanalytics -c "DELETE FROM auditoria"
ERROR:  La auditoria es de solo insercion: no se permite DELETE
CONTEXT:  PL/pgSQL function trg_auditoria_inmutable() line 3 at RAISE

C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>
```

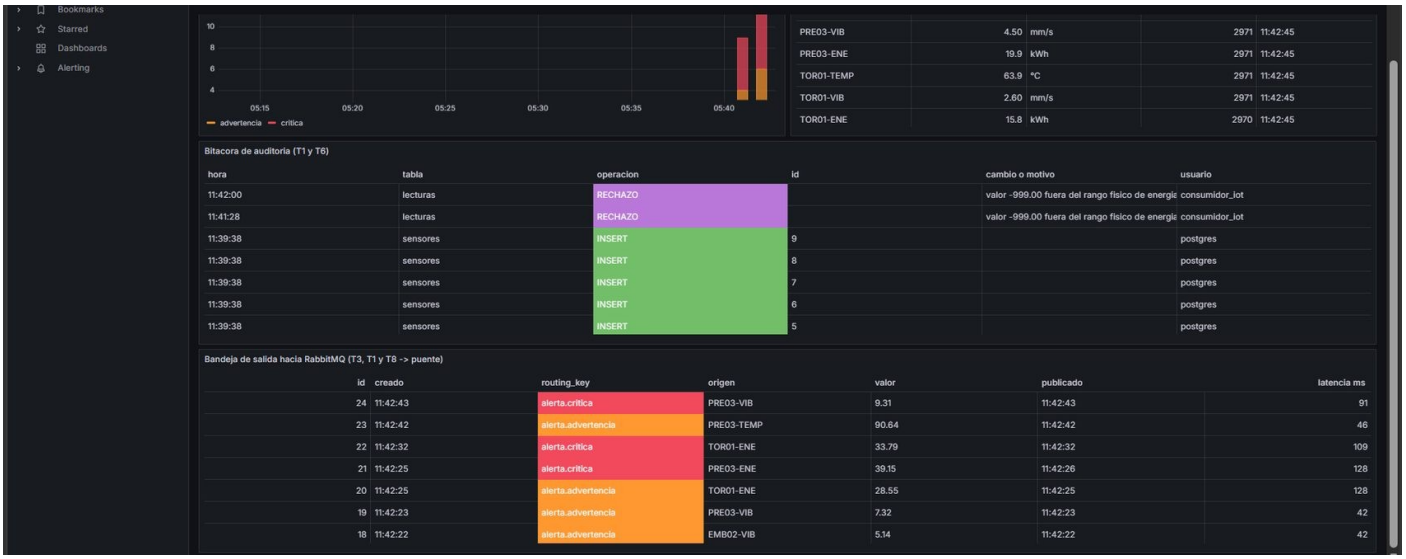
Auditoría de cambios y bitácora inmutable.

Evidencia 5 — P9: el tablero de Grafana muestra lo que automatizan los disparadores: 17 alertas, 2 lecturas descartadas, 19 eventos publicados en RabbitMQ y 0 pendientes; además, el estado en vivo y la bitácora de auditoría.



Tablero de los disparadores.

Evidencia 6 — P9: la bandeja de salida muestra cada evento con su routing key, la hora en que se creó y en que el puente lo publicó: entre 42 y 128 ms después.



Bandeja de salida hacia RabbitMQ.

Evidencia 7 — P9: con TOR-01 en mantenimiento, el notificador recibe en tiempo real cada lectura rechazada por T1; el puente lleva 241 eventos publicados y el consumidor solo registra que el disparador descartó la lectura.

```
C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>
C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>docker compose logs --tail 12 notificador
triggers-notificador  | [RECHAZADA] TOR01-VIB = 4.45 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-ENE = 18.31 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-TEMP = 82.46 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-VIB = 3.21 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-ENE = 20.0 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-TEMP = 69.2 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-VIB = 3.99 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-ENE = 16.58 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-TEMP = 68.07 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-VIB = 3.85 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-ENE = 15.08 -> dispositivo TOR-01 en mantenimiento
triggers-notificador  | [RECHAZADA] TOR01-TEMP = 60.6 -> dispositivo TOR-01 en mantenimiento

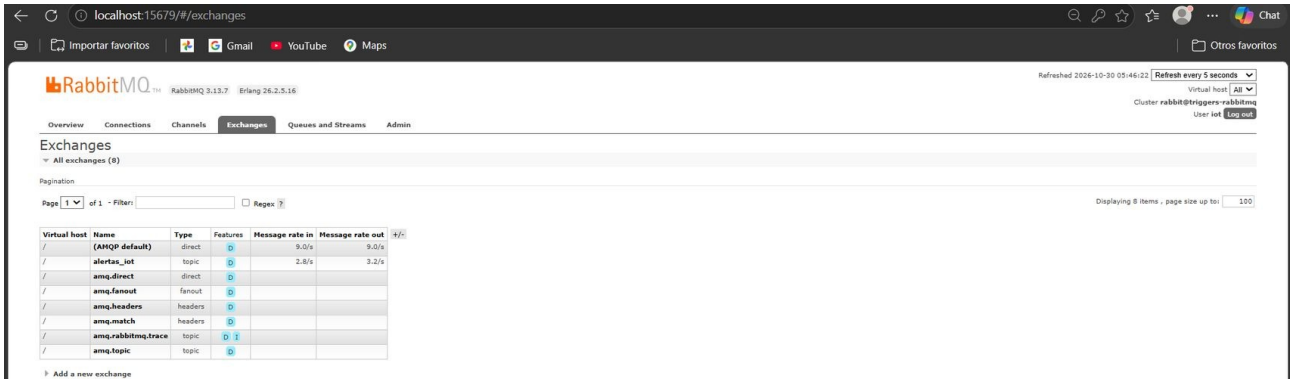
C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>docker compose logs --tail 5 puente
triggers-puente  | Publicados 3 eventos en 'alertas_iot' (total 228)
triggers-puente  | Publicados 3 eventos en 'alertas_iot' (total 231)
triggers-puente  | Publicados 3 eventos en 'alertas_iot' (total 234)
triggers-puente  | Publicados 3 eventos en 'alertas_iot' (total 237)
triggers-puente  | Publicados 4 eventos en 'alertas_iot' (total 241)

C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>docker compose logs --tail 5 consumidor
triggers-consumidor  | Descartada por t1_lecturas_validar: TOR01-VIB = 3.27
triggers-consumidor  | Descartada por t1_lecturas_validar: TOR01-ENE = 15.87
triggers-consumidor  | Descartada por t1_lecturas_validar: TOR01-TEMP = 66.03
triggers-consumidor  | Descartada por t1_lecturas_validar: TOR01-VIB = 3.97
triggers-consumidor  | Descartada por t1_lecturas_validar: TOR01-ENE = 16.9

C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>
```

Notificador, puente y consumidor trabajando en cadena.

Evidencia 8 — P9: el exchange alertas_iot de tipo topic. Sale más de lo que entra (3.2 contra 2.8 mensajes por segundo) porque cada alerta crítica se copia a dos colas: notificaciones y solo_criticas.



Exchange topic alertas_iot en RabbitMQ.

7. Análisis de rendimiento

Se insertaron 10,000 lecturas normales (sin alertas) en una sola sentencia, dentro de una transacción que después se deshizo, para medir el costo de los disparadores en tres escenarios:

ESCENARIO	TIEMPO	COSTO POR LECTURA
F1. Sin disparadores	426.8 ms	0.043 ms
F2. Con disparadores (T5 por sentencia)	1,570.6 ms	0.157 ms
F3. Con disparadores (T5 reescrito por fila)	3,331.5 ms	0.333 ms



Medición de rendimiento con \timing.

Interpretación: automatizar validación, alertas y estado cuesta unos 0.11 ms por lectura. Al ritmo real del sistema (9 lecturas por segundo) son menos de 1 ms por segundo de trabajo, un costo despreciable a cambio de que ninguna lectura escape a las reglas. La decisión de diseño que más impacto tuvo fue T5: la misma lógica escrita por fila tarda **2.1 veces más**, porque hace 10,000 actualizaciones de `estado_sensor` sobre las mismas 9 filas, mientras que la versión por sentencia, con tabla de transición, hace una sola consulta que agrupa todas las lecturas. Las pruebas en un equipo de desarrollo dieron la misma proporción (179 ms contra 624 ms).

También se cuidó el costo fuera del disparador: la publicación en RabbitMQ no ocurre dentro de la transacción, así que una caída de RabbitMQ no frena el registro de lecturas, y `pg_notify` se ejecuta una vez por sentencia, no por cada evento.

7.1 Impacto en la gestión de datos IoT

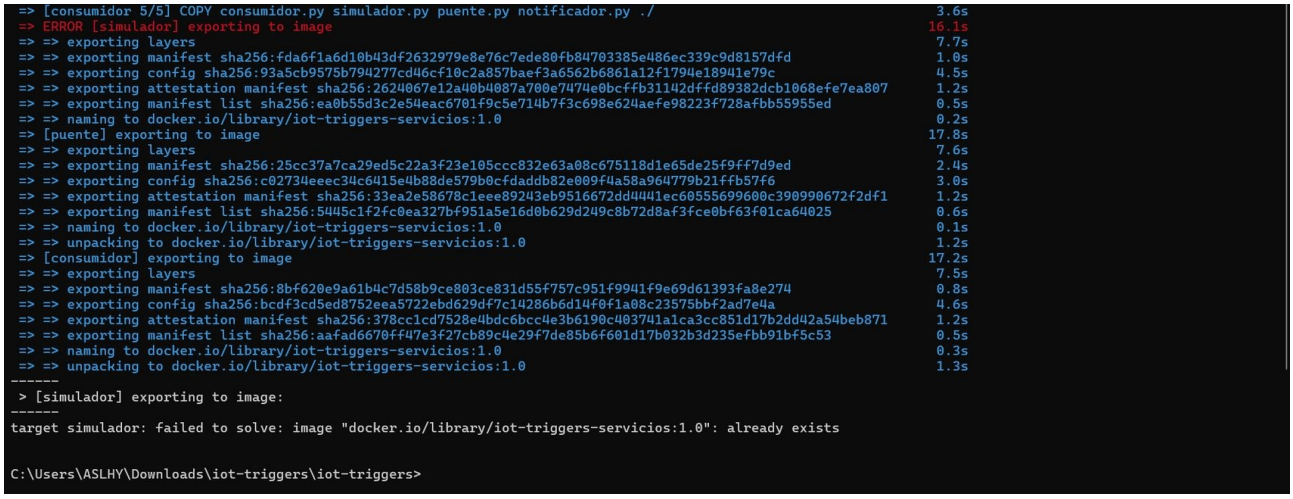
ASPECTO	ANTES (LÓGICA EN LA APLICACIÓN)	CON DISPARADORES
Integridad	Depende de que cada cliente valide	Ninguna lectura imposible entra, venga de donde venga
Trazabilidad	No hay registro de cambios	Cada cambio queda con antes, después, usuario y hora, sin poder borrarse
Alertas en tiempo real	Consultas periódicas	Notificación por RabbitMQ en milisegundos, sin perder eventos
Consistencia	Alertas abiertas de equipos en mantenimiento	T8 las cierra automáticamente
Simplicidad del cliente	Validar, alertar y auditar en cada servicio	Un solo INSERT

8. Problemas encontrados y solución

8.1 Construcción simultánea de la misma imagen

Al levantar el entorno, Docker falló con “image iot-triggers-servicios:1.0 already exists”. El consumidor, el simulador y el puente tenían `build` y el mismo nombre de imagen; Compose los construyó en paralelo y, cuando el segundo terminó, la imagen ya había sido registrada por el

primero. En la Tarea 8 solo dos servicios compartían imagen y no coincidieron, por eso no había aparecido.



Error al construir la misma imagen en paralelo.

Solución: quitar el nombre de imagen compartido y dejar que cada servicio construya la suya (`iot-triggers-consumidor`, `iot-triggers-puente`, etc.). Las capas son idénticas, así que Docker las reutiliza de la caché y la construcción no tarda más.

8.2 Prueba de rendimiento sesgada

La primera versión de la prueba insertaba el valor 10 en todos los sensores. Ese valor supera el umbral de los sensores de vibración (5 a 7 mm/s), así que la prueba también generaba miles de alertas y eventos, y la comparación no medía solo el disparador T5. Se corrigió usando el 80 % del umbral de cada sensor, para que todas las lecturas fueran normales.

8.3 La auditoría podía vaciarse con TRUNCATE

Al revisar la protección de la bitácora se notó que `TRUNCATE` no ejecuta los disparadores por fila, así que T9 no lo impedía. Se agregó un disparador `BEFORE TRUNCATE ... FOR EACH STATEMENT` con la misma función.

8.4 Prevención

El error de RabbitMQ con `.erlang.cookie` de las Tareas 5 y 7 no se repitió, porque su solución viene incluida en `docker-compose.yml`.

9. Control de versiones y colaboración

La actividad se desarrolló de forma individual. El proyecto se publicó en un repositorio público con un README que explica cada disparador y cómo levantar el entorno: https://github.com/luisgael-ml/iot-triggers

```
git init
git config user.name "Luis Gael Monterrubio López"
git config user.email "monterrubiolopezluisgael@gmail.com"
git add .
git commit -m "Disparadores en PostgreSQL para IoT Analytics Dashboard"
git branch -M main
git remote add origin https://github.com/luisgael-ml/iot-triggers.git
git push -u origin main
```

Evidencia 9: el commit se registró a nombre de Luis Gael Monterrubio López y se publicó en GitHub.

```
C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>git commit -m "Disparadores en PostgreSQL para IoT Analytics Dashboard"
[main (root-commit) 2f34ac7] Disparadores en PostgreSQL para IoT Analytics Dashboard
 14 files changed, 1420 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 datos/init.sql
 create mode 100644 datos/pruebas.sql
 create mode 100644 docker-compose.yml
 create mode 100644 grafana/dashboards/iot-triggers.json
 create mode 100644 grafana/provisioning/dashboards/dashboards.yml
 create mode 100644 grafana/provisioning/datasources/postgres.yml
 create mode 100644 servicios/Dockerfile
 create mode 100644 servicios/consumidor.py
 create mode 100644 servicios/notificador.py
 create mode 100644 servicios/puente.py
 create mode 100644 servicios/requirements.txt
 create mode 100644 servicios/simulador.py

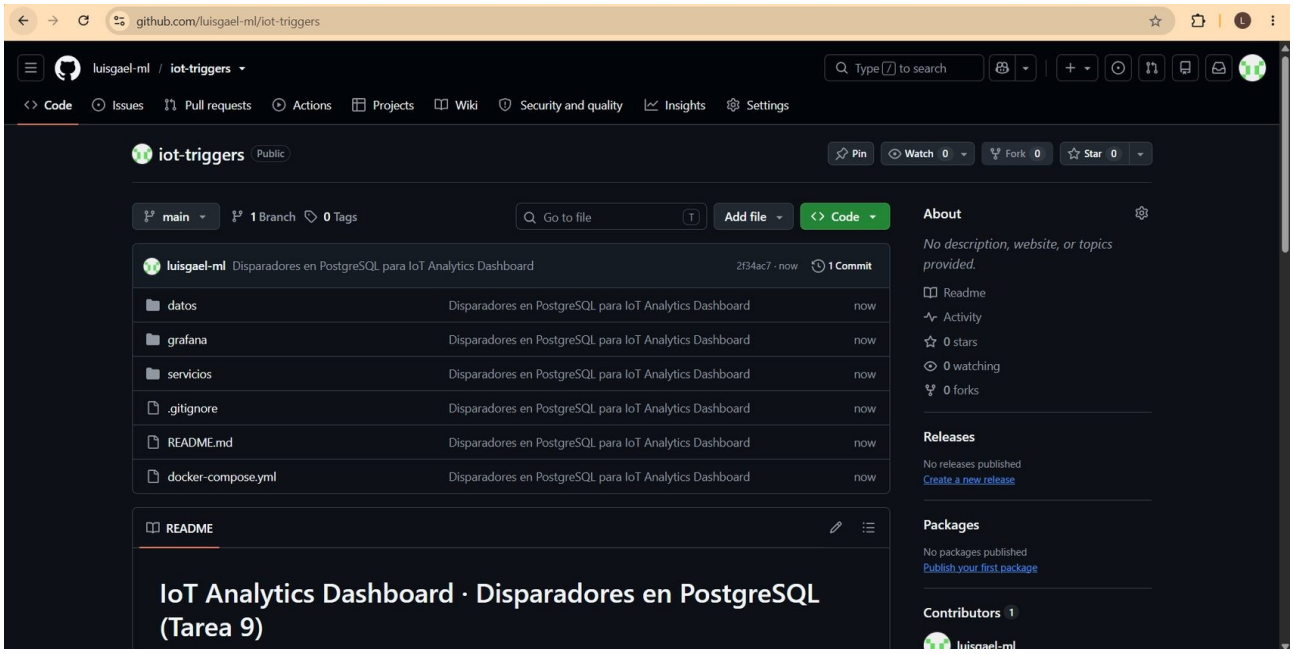
C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>git branch -M main

C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>git remote add origin https://github.com/luisgael-ml/iot-triggers.git

C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>git push -u origin main
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 4 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (23/23), 16.10 KiB | 433.00 KiB/s, done.
Total 23 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/luisgael-ml/iot-triggers.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\ASLHY\Downloads\iot-triggers\iot-triggers>
```

Publicación del proyecto en GitHub.



Repositorio iot-triggers.

9.1 Reflexión individual sobre la colaboración

En las entrevistas de las Tareas 7 y 8 el CTO me dio retroalimentación sobre escalabilidad, y la apliqué aquí: el disparador de estado se diseñó por sentencia y el puente usa `FOR UPDATE SKIP LOCKED`, lo que permite levantar varios puentes en paralelo, la misma idea de escalar consumidores que propuse en la entrevista anterior.

Pensando en ayudar a otros a crear disparadores, dejé el proyecto preparado para que sirva de referencia: el README explica cada disparador con su tabla, evento, tipo y propósito; los nombres llevan un prefijo numérico que hace visible el orden de ejecución; la función de auditoría es genérica, así que un compañero puede auditar una tabla nueva con una sola línea `CREATE TRIGGER`; y el archivo de pruebas reproduce cada caso y la comparación de rendimiento. También documenté los

tres problemas y el error de diseño que corregí en la prueba de rendimiento, para que otros eviten las mismas trampas, en especial que TRUNCATE no activa los disparadores por fila.

10. Conclusiones

Los disparadores convirtieron la base de datos del IoT Analytics Dashboard en un sistema que reacciona por sí mismo a los eventos. Con un solo INSERT del consumidor, la lectura se valida, genera su alerta, actualiza el estado del sensor y, si es necesario, notifica a RabbitMQ, sin que ninguna aplicación tenga que repetir esas reglas.

La tabla de auditoría registra automáticamente los datos fuera de rango y todos los cambios en los datos de los sensores, con el antes, el después y el usuario real, y no se puede alterar. Para las alertas en tiempo real, el patrón outbox con LISTEN/NOTIFY resolvió la limitación de que un disparador no puede comunicarse con sistemas externos: las alertas llegaron a RabbitMQ en milisegundos, solo después de confirmarse la transacción y sin riesgo de perderse.

Elegir bien el tipo de disparador también importa: la versión por sentencia con tabla de transición fue 2.1 veces más rápida que la versión por fila. Con un costo de alrededor de 0.11 ms por lectura, el módulo garantiza integridad, trazabilidad y notificación inmediata, cumpliendo el objetivo de NexoDev de un manejo de datos IoT íntegro y eficiente.

Referencias

Docker Inc. (s.f.). *Build images with Compose*. Docker Docs. https://docs.docker.com/compose/how-tos/use-build/

PostgreSQL Global Development Group. (s.f.). *CREATE TRIGGER*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-createtrigger.html

PostgreSQL Global Development Group. (s.f.). *NOTIFY*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-notify.html

PostgreSQL Global Development Group. (s.f.). *Overview of trigger behavior*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/trigger-definition.html

PostgreSQL Global Development Group. (s.f.). *Trigger functions (PL/pgSQL)*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/plpgsql-trigger.html

Psycopg. (s.f.). *Asynchronous notifications*. Psycopg 3 documentation. https://www.psycopg.org/psycopg3/docs/advanced/async.html#asynchronous-notifications

RabbitMQ. (s.f.). *Consumer acknowledgements and publisher confirms*. RabbitMQ documentation. https://www.rabbitmq.com/docs/confirms

RabbitMQ. (s.f.). *Exchanges (topic exchange)*. RabbitMQ documentation. https://www.rabbitmq.com/docs/exchanges

Richardson, C. (s.f.). *Pattern: Transactional outbox*. Microservices.io. https://microservices.io/patterns/data/transactional-outbox.html